

API Endpoints TODO - App Balloo Messenger

Полное техническое задание на разработку API для мессенджера App Balloo.

Важные примечания по архитектуре

База Данных

- **НЕ отдельный сервер PostgreSQL**
- База данных должна быть поднята **на том же домене и процессе**, что и API
- Использовать встроенную БД или процесс-локальное хранилище (SQLite, Better-SQLite3, или встраиваемая PostgreSQL)
- Все данные должны храниться на одном сервере

Безопасность и Шифрование

- **E2E шифрование**: Все сообщения шифруются на клиенте (AES-256-GCM)
- **RSA-2048**: Обмен ключами между пользователями
- **Ключи шифрования НЕ хранятся на сервере** в зашифрованном виде
- Сервер передаёт только зашифрованные данные
- Поддержка **шифрования файлов** перед загрузкой

Яндекс.Интеграции

- **Яндекс.Авторизация**: OAuth 2.0 вход через Яндекс
- **Яндекс.Диск**: Хранение файлов и медиа на Яндекс.Диске пользователя
- API ключи и секреты Яндекс хранятся только на сервере
- Токены пользователей шифруются перед сохранением

Содержание

1. [Архитектура и Безопасность](#)
2. [Аутентификация](#)
3. [Яндекс.Авторизация](#)
4. [Пользователи](#)
5. [Чаты](#)
6. [Сообщения](#)
7. [Контакты](#)
8. [Файлы и Яндекс.Диск](#)
9. [Уведомления](#)
10. [Группы и Роли](#)
11. [Приглашения](#)
12. [Баллы и Награды](#)
13. [История Сообщений](#)
14. [Отчёты и Модерация](#)

15. [Администрирование](#)
16. [Синхронизация](#)
17. [WebRTC и Звонки](#)
18. [Статусы \(Сторис\)](#)
19. [Поиск](#)
20. [Инсталлер и Обновления](#)
21. [WebSocket События](#)

Архитектура и Безопасность

Общие требования

База Данных:

- Использовать встроенную/встраиваемую БД (SQLite, NeDB, или embedded PostgreSQL)
- Все данные на одном сервере с API
- Автоматическое создание и миграции схем при старте

Шифрование:

- Все сообщения передаются в зашифрованном виде (E2E)
- Сервер не имеет доступа к ключам расшифровки
- Файлы шифруются перед загрузкой на Яндекс.Диск
- Ключи шифрования хранятся только на клиентах

Яндекс.Интеграции:

- Client ID и Client Secret хранятся только на сервере в .env
- Токены пользователей шифруются перед сохранением в БД
- Рефреш токены для автоматического обновления доступа к Диску

Аутентификация

POST `/api/v1/auth/register`

Регистрация нового пользователя по email

Request Body:

```
{
  "email": "string (email, required)",
  "password": "string (min 8 chars, required)",
  "displayName": "string (required)",
  "fullName": "string (optional)",
  "birthDate": "number (timestamp, optional)",
  "phone": "string (optional)"
}
```

Response: 201 Created

```
{
  "success": true,
  "data": {
    "user": {
      "id": "uuid",
      "email": "string",
      "displayName": "string",
      "fullName": "string",
      "avatar": "string (url)",
      "publicKey": "string (base64, RSA public key)",
      "createdAt": 1234567890,
      "provider": "email"
    },
    "accessToken": "jwt_token",
    "refreshToken": "refresh_token"
  }
}
```

POST /api/v1/auth/login

Вход по email и паролю

Request Body:

```
{
  "email": "string (email, required)",
  "password": "string, required",
  "deviceInfo": {
    "platform": "web|android|ios|desktop",
    "deviceId": "string",
    "pushToken": "string (optional)"
  }
}
```

Response: 200 OK

POST /api/v1/auth/logout

Выход из системы

Headers: Authorization: Bearer <token>

Query: ?allDevices=true (выйти со всех устройств)

Response: 200 OK

POST /api/v1/auth/refresh

Обновление access токена

Request Body:

```
{
  "refreshToken": "string"
}
```

Response: 200 OK

POST /api/v1/auth/forgot-password

Запрос на восстановление пароля (отправка кода на email)

Request Body:

```
{
  "email": "string (email, required)"
}
```

Response: 200 OK

POST /api/v1/auth/verify-code

Подтверждение кода (для восстановления пароля, регистрации)

Request Body:

```
{
  "email": "string, required",
  "code": "string (6 digits), required",
  "type": "password_reset|registration|email_verification"
}
```

Response: 200 OK

POST /api/v1/auth/reset-password

Сброс пароля после подтверждения кода

Request Body:

```
{
  "email": "string, required",
  "code": "string, required",
}
```

```
"newPassword": "string (min 8 chars), required"
}
```

Response: 200 OK

GET /api/v1/auth/me

Получение данных текущего пользователя

Headers: Authorization: Bearer <token>

Response: 200 OK

PUT /api/v1/auth/change-password

Смена пароля (требуется старый пароль)

Headers: Authorization: Bearer <token>

Request Body:

```
{
  "oldPassword": "string, required",
  "newPassword": "string (min 8 chars), required"
}
```

GET /api/v1/auth/sessions

Получение списка активных сессий

Headers: Authorization: Bearer <token>

Response: 200 OK

```
{
  "success": true,
  "data": {
    "sessions": [
      {
        "id": "uuid",
        "platform": "web",
        "deviceId": "string",
        "lastActive": 1234567890,
        "isCurrent": true
      }
    ]
  }
}
```

DELETE `/api/v1/auth/sessions/:sessionId`

Завершение сессии

Яндекс.Авторизация

GET `/api/v1/auth/yandex/authorize`

Перенаправление на Яндекс.OAuth для авторизации

Query Parameters:

- `redirect_uri`: string (callback URL)
- `response_type`: "code"
- `client_id`: string (Yandex Client ID)
- `state`: string (CSRF token)

POST `/api/v1/auth/yandex/callback`

Callback от Яндекса после авторизации

Request Body:

```
{
  "code": "string (authorization code from Yandex)",
  "redirect_uri": "string",
  "platform": "web|android|ios|desktop",
  "deviceId": "string",
  "pushToken": "string (optional)"
}
```

Response: 200 OK

```
{
  "success": true,
  "data": {
    "user": { ... },
    "accessToken": "jwt_token",
    "refreshToken": "refresh_token",
    "isNewUser": true
  }
}
```

POST `/api/v1/auth/yandex/link`

Привязка Яндекс.аккаунта к существующему email-аккаунту

Headers: Authorization: Bearer <token>

Request Body:

```
{
  "code": "string",
  "redirect_uri": "string"
}
```

POST /api/v1/auth/yandex/unlink

Отвязка Яндекс.аккаунта

Headers: Authorization: Bearer <token>

GET /api/v1/auth/yandex/status

Проверка статуса подключения Яндекс.аккаунта

Headers: Authorization: Bearer <token>

Response: 200 OK

```
{
  "success": true,
  "data": {
    "isConnected": true,
    "yandexId": "string",
    "yandexName": "string",
    "yandexAvatar": "string"
  }
}
```

Пользователи

GET /api/v1/users/:userId

Получение публичных данных пользователя

Response: 200 OK

```
{
  "success": true,
  "data": {
    "id": "uuid",
    "displayName": "string",
  }
}
```

```

    "fullName": "string",
    "avatar": "string",
    "publicKey": "string (RSA public key)",
    "status": "online|offline|away|busy",
    "lastSeen": 1234567890,
    "isOnline": true,
    "bio": "string",
    "birthDate": 1234567890,
    "familyRelations": [
      {
        "userId": "uuid",
        "relation": "parent|sibling|child|spouse",
        "name": "string"
      }
    ]
  }
}

```

GET `/api/v1/users/search`

Поиск пользователей

Query Parameters:

- `q`: string (search query, required)
- `limit`: number (default: 20, max: 100)
- `offset`: number (default: 0)

Response: 200 OK

PUT `/api/v1/users/me`

Обновление данных текущего пользователя

Headers: `Authorization: Bearer <token>`

Request Body:

```

{
  "displayName": "string",
  "fullName": "string",
  "bio": "string",
  "birthDate": 1234567890,
  "phone": "string",
  "familyRelations": [...],
  "settings": {
    "theme": "light|dark",
    "language": "ru|en|hi|zh|tt",
    "notificationsEnabled": true,
    "soundEnabled": true,
    "vibrateEnabled": true
  }
}

```



```
}  
}
```

PUT `/api/v1/users/me/avatar`

Загрузка аватара

Headers: `Authorization: Bearer <token>`

Content-Type: `multipart/form-data`

Request:

- `file`: File (image)

Response: `200 OK`

PUT `/api/v1/users/me/status`

Обновление статуса

Request Body:

```
{  
  "status": "online|offline|away|busy",  
  "customStatus": "string"  
}
```

GET `/api/v1/users/me/contacts`

Получение списка контактов пользователя

Headers: `Authorization: Bearer <token>`

GET `/api/v1/users/me/devices`

Получение списка устройств

Headers: `Authorization: Bearer <token>`

Response: `200 OK`

PUT `/api/v1/users/me/devices/:deviceId`

Обновление информации об устройстве

Request Body:

```
{  
  "pushToken": "string",
```

```
"platform": "string",  
"deviceName": "string"  
}
```

DELETE /api/v1/users/me/devices/:deviceId

Удаление устройства

Чаты

GET /api/v1/chats

Получение списка чатов пользователя

Headers: Authorization: Bearer <token>

Query Parameters:

- limit: number
- offset: number
- type: "private" | "group"
- filter: "favorite" | "pinned" | "unread"

Response: 200 OK

```
{  
  "success": true,  
  "data": {  
    "chats": [  
      {  
        "id": "uuid",  
        "type": "private|group",  
        "name": "string",  
        "avatar": "string",  
        "participants": ["uuid", ...],  
        "lastMessage": { ... },  
        "unreadCount": 0,  
        "isFavorite": true,  
        "pinned": true,  
        "muted": false,  
        "updatedAt": 1234567890  
      }  
    ],  
    "pagination": {  
      "total": 50,  
      "limit": 20,  
      "offset": 0  
    }  
  }  
}
```

```
}  
}
```

GET /api/v1/chats/:chatId

Получение информации о чате

Headers: Authorization: Bearer <token>

Response: 200 OK

POST /api/v1/chats

Создание нового чата

Headers: Authorization: Bearer <token>

Request Body (private):

```
{  
  "type": "private",  
  "participantIds": ["uuid"], // один пользователь  
  "encryptedKeys": {  
    "sharedKey": "string (AES key)",  
    "recipientPublicKey": "string (RSA public key)"  
  }  
}
```

Request Body (group):

```
{  
  "type": "group",  
  "name": "string",  
  "avatar": "string (optional)",  
  "description": "string",  
  "participantIds": ["uuid", ...],  
  "adminIds": ["uuid", ...]  
}
```

Response: 201 Created

PUT /api/v1/chats/:chatId

Обновление чата (для групп)

Headers: Authorization: Bearer <token>

Request Body:

```
{
  "name": "string",
  "avatar": "string",
  "description": "string"
}
```

DELETE /api/v1/chats/:chatId

Удаление/выход из чата

Headers: Authorization: Bearer <token>

Query: ?deleteForEveryone=true (для создателя группы)

PUT /api/v1/chats/:chatId/favorite

Добавить/удалить чат в избранное

Headers: Authorization: Bearer <token>

Request Body:

```
{
  "isFavorite": true
}
```

PUT /api/v1/chats/:chatId/pin

Закрепить/открепить чат

Request Body:

```
{
  "pinned": true
}
```

PUT /api/v1/chats/:chatId/mute

Отключить/включить уведомления

Request Body:

```
{
  "muted": true,
```

```
"muteUntil": 1234567890 // optional, timestamp
}
```

GET `/api/v1/chats/:chatId/members`

Получение списка участников чата

Response: 200 OK

POST `/api/v1/chats/:chatId/members`

Добавление участника в чат

Headers: Authorization: Bearer <token>

Request Body:

```
{
  "userId": "uuid",
  "role": "creator|moderator|author|reader" // для групп
}
```

DELETE `/api/v1/chats/:chatId/members/:userId`

Удаление участника из чата

PUT `/api/v1/chats/:chatId/members/:userId/role`

Изменение роли участника

Request Body:

```
{
  "role": "creator|moderator|author|reader"
}
```

PUT `/api/v1/chats/:chatId/read`

Отметить чат как прочитанный

Request Body:

```
{
  "lastMessageId": "uuid"
}
```

POST /api/v1/chats/:chatId/typing

Отправить событие "печатает..."

Headers: Authorization: Bearer <token>

Request Body:

```
{
  "isTyping": true
}
```

Сообщения

GET /api/v1/chats/:chatId/messages

Получение истории сообщений

Headers: Authorization: Bearer <token>

Query Parameters:

- **limit:** number (default: 50, max: 100)
- **before:** string (messageld, get messages before this)
- **after:** string (messageld, get messages after this)

Response: 200 OK

```
{
  "success": true,
  "data": {
    "messages": [
      {
        "id": "uuid",
        "chatId": "uuid",
        "senderId": "uuid",
        "sender": { ... },
        "type": "text|image|video|file|audio|document|system",
        "content": "string (ENCRYPTED)",
        "encryptedInfo": {
          "ciphertext": "string",
          "iv": "string",
          "authTag": "string",
          "keyId": "string"
        },
        "attachmentId": "uuid",
        "attachment": { ... },
        "replyToId": "uuid",
      }
    ]
  }
}
```

```

    "replyToMessage": { ... },
    "forwardFromId": "uuid",
    "reactions": { ... },
    "reactionsCount": { "👍": 5, "❤️": 2 },
    "readBy": ["uuid", ...],
    "isRead": false,
    "isEdited": false,
    "editedAt": 1234567890,
    "createdAt": 1234567890,
    "status": "sending|sent|delivered|read"
  }
],
"hasMore": true
}

```

POST `/api/v1/chats/:chatId/messages`

Отправка сообщения

Headers: `Authorization: Bearer <token>`

Content-Type: `multipart/form-data` или `application/json`

Request Body (text, encrypted):

```

{
  "content": "string (ENCRYPTED)",
  "encryptedInfo": {
    "ciphertext": "string",
    "iv": "string",
    "authTag": "string",
    "keyId": "string"
  },
  "type": "text",
  "replyToId": "uuid (optional)",
  "forwardFromId": "uuid (optional)",
  "forwardFromChatId": "uuid (optional)"
}

```

Request Body (с вложением):

```

{
  "content": "string (optional, can be encrypted)",
  "type": "image|video|file|audio",
  "attachmentId": "uuid (from upload)",
  "replyToId": "uuid"
}

```

Response: 201 Created

PUT /api/v1/messages/:messageId

Редактирование сообщения

Headers: Authorization: Bearer <token>

Request Body:

```
{
  "content": "string (ENCRYPTED)",
  "encryptedInfo": { ... }
}
```

Response: 200 OK

DELETE /api/v1/messages/:messageId

Удаление сообщения

Headers: Authorization: Bearer <token>

Query: ?deleteForEveryone=true

Response: 200 OK

POST /api/v1/messages/:messageId/reactions

Добавление реакции к сообщению

Headers: Authorization: Bearer <token>

Request Body:

```
{
  "emoji": "👍|👎|❤️|😄|🙌|💧|😂|😱|😬|👏|🤔|😎|😏|🥰|🥳|100"
}
```

Response: 200 OK

DELETE /api/v1/messages/:messageId/reactions/:emoji

Удаление реакции

PUT /api/v1/messages/:messageId/read

Подтверждение прочтения сообщения

Headers: Authorization: Bearer <token>

Контакты

GET /api/v1/contacts

Получение списка контактов

Headers: Authorization: Bearer <token>

Query Parameters:

- search: string
- isFavorite: boolean
- limit: number
- offset: number

Response: 200 OK

POST /api/v1/contacts

Добавление контакта

Headers: Authorization: Bearer <token>

Request Body:

```
{
  "userId": "uuid",
  "displayName": "string (optional, custom name)"
}
```

DELETE /api/v1/contacts/:userId

Удаление контакта

PUT /api/v1/contacts/:userId/favorite

Добавить/удалить из избранных

Request Body:

```
{
  "isFavorite": true
}
```

PUT /api/v1/contacts/:userId/block

Заблокировать/разблокировать контакт

Request Body:

```
{
  "isBlocked": true
}
```

GET `/api/v1/contacts/requests`

Получение запросов в контакты

Headers: `Authorization: Bearer <token>`

Query Parameters:

- `type: "received" | "sent"`

POST `/api/v1/contacts/requests`

Отправка запроса в контакты

Request Body:

```
{
  "userId": "uuid",
  "message": "string (optional)"
}
```

PUT `/api/v1/contacts/requests/:requestId`

Обработка запроса

Request Body:

```
{
  "action": "accept|reject"
}
```

Файлы и Яндекс.Диск

POST `/api/v1/files/upload`

Загрузка файла (на Яндекс.Диск пользователя)

Headers: Authorization: Bearer <token>

Content-Type: multipart/form-data

Request:

- **file:** File (required)
- **type:** "image|video|audio|document" (optional)
- **chatId:** "uuid" (optional, для связки с чатом)

Response: 201 Created

```
{
  "success": true,
  "data": {
    "id": "uuid",
    "fileName": "string",
    "mimeType": "string",
    "size": 12345,
    "yandexDiskPath": "/messenger/uploads/uuid",
    "yandexDiskId": "string",
    "thumbnailUrl": "string (optional)",
    "width": 1920,
    "height": 1080,
    "duration": 120,
    "createdAt": 1234567890,
    "status": "ready"
  }
}
```

GET /api/v1/files/:fileId

Получение информации о файле

GET /api/v1/files/:fileId/download

Скачивание файла

Response: 200 OK (file stream)

DELETE /api/v1/files/:fileId

Удаление файла

POST /api/v1/files/encrypt

Шифрование файла перед загрузкой (helper для клиента)

Headers: Authorization: Bearer <token>

Content-Type: multipart/form-data

Request:

- **file**: File
- **encryptionKey**: "string (AES key)"

Response: 200 OK

GET /api/v1/disk/files

Получение списка файлов на Яндекс.Диске

Headers: Authorization: Bearer <token>**Query Parameters:**

- **path**: string (path on Yandex Disk, default: "/messenger")
- **limit**: number

Response: 200 OK

POST /api/v1/disk/files

Загрузка файла на Яндекс.Диск (прямая)

DELETE /api/v1/disk/files/:path

Удаление файла с Яндекс.Диска

POST /api/v1/disk/files/:path/share

Получение публичной ссылки на файл

Response: 200 OK

```
{
  "success": true,
  "data": {
    "publicUrl": "https://yadi.sk/..."
  }
}
```

GET /api/v1/disk/quota

Получение информации о месте на Яндекс.Диске

Response: 200 OK

POST /api/v1/disk/connect

Подключение Яндекс.Диска (OAuth flow)

Request Body:

```
{
  "code": "string (authorization code)",
  "redirectUri": "string"
}
```

POST `/api/v1/disk/disconnect`

Отключение Яндекс.Диска

Headers: `Authorization: Bearer <token>`

Уведомления

GET `/api/v1/notifications`

Получение уведомлений пользователя

Headers: `Authorization: Bearer <token>`

Query Parameters:

- `type`: "message|system|friend|invite|admin"
- `read`: boolean
- `limit`: number
- `offset`: number

Response: 200 OK

```
{
  "success": true,
  "data": {
    "notifications": [
      {
        "id": "uuid",
        "type": "message|system|friend|invite|admin",
        "title": "string",
        "body": "string",
        "icon": "string",
        "url": "/chats/uuid",
        "data": {},
        "read": false,
        "createdAt": 1234567890
      }
    ],
    "unreadCount": 5
  }
}
```

```
}  
}
```

PUT `/api/v1/notifications/:notificationId/read`

Отметить уведомление как прочитанное

PUT `/api/v1/notifications/read-all`

Отметить все уведомления как прочитанные

DELETE `/api/v1/notifications/:notificationId`

Удалить уведомление

POST `/api/v1/notifications/register-token`

Регистрация push-токена

Headers: `Authorization: Bearer <token>`

Request Body:

```
{  
  "pushToken": "string",  
  "platform": "web|android|ios|desktop"  
}
```

Группы и Роли

POST `/api/v1/groups`

Создание группы

Headers: `Authorization: Bearer <token>`

Request Body:

```
{  
  "name": "string",  
  "description": "string",  
  "avatar": "string (optional)",  
  "participantIds": ["uuid", ...],  
  "adminIds": ["uuid", ...],  
  "settings": {  
    "onlyAdminsCanPost": false,  
    "onlyAdminsCanAddMembers": false,  
  },  
}
```

```
    "allowReadersToInvite": false
  }
}
```

Response: 201 Created

GET /api/v1/groups/:groupId

Получение информации о группе

PUT /api/v1/groups/:groupId

Обновление группы

DELETE /api/v1/groups/:groupId

Удаление группы (только создатель)

PUT /api/v1/groups/:groupId/settings

Обновление настроек группы

Request Body:

```
{
  "onlyAdminsCanPost": true,
  "onlyAdminsCanAddMembers": true
}
```

GET /api/v1/groups/:groupId/permissions

Получение прав участников группы

PUT /api/v1/groups/:groupId/permissions/:userId

Изменение прав участника

Request Body:

```
{
  "role": "creator|moderator|author|reader"
}
```

POST /api/v1/groups/:groupId/transfer-ownership

Передача прав создателя

Request Body:

```
{
  "newOwnerId": "uuid"
}
```

✉ Приглашения

GET /api/v1/invitations

Получение списка активных приглашений

Headers: Authorization: Bearer <token>

Query Parameters:

- chatId: "uuid" (filter by chat)
- isActive: boolean

POST /api/v1/invitations

Создание приглашения

Headers: Authorization: Bearer <token>

Request Body:

```
{
  "chatId": "uuid",
  "isPermanent": false,
  "maxUses": 10,
  "expiresAt": 1234567890,
  "message": "string (optional, custom message)"
}
```

Response: 201 Created

```
{
  "success": true,
  "data": {
    "id": "uuid",
    "code": "unique-code",
    "inviteUrl": "https://messenger.app/invite/unique-code",
    "expiresAt": 1234567890,
    "maxUses": 10,
    "usedCount": 0
  }
}
```



```
}  
}
```

GET `/api/v1/invitations/:code`

Получение информации о приглашении (публично)

Response: 200 OK

POST `/api/v1/invitations/:code/accept`

Принять приглашение (присоединиться к чату)

Request Body:

```
{  
  "code": "string"  
}
```

Response: 200 OK

DELETE `/api/v1/invitations/:invitationId`

Отменить/удалить приглашение

PUT `/api/v1/invitations/:invitationId/revoke`

Отозвать приглашение

★ Баллы и Награды

GET `/api/v1/points`

Получение баланса баллов пользователя

Headers: Authorization: Bearer <token>

Response: 200 OK

```
{  
  "success": true,  
  "data": {  
    "userId": "uuid",  
    "points": 100,  
    "totalEarned": 200,  
    "totalSpent": 100,  
    "inviteCount": 5,  
  }  
}
```

```
"history": [
  {
    "id": "uuid",
    "type": "invite_bonus|spent|bonus",
    "amount": 2,
    "description": "Приглашение пользователя",
    "createdAt": 1234567890,
    "relatedInviteId": "uuid"
  }
]
```

История Сообщений

GET `/api/v1/history`

Получение истории сообщений пользователя (все чаты)

Headers: `Authorization: Bearer <token>`

Query Parameters:

- `search`: string (поиск по контенту)
- `type`: "text|image|video|file|audio"
- `from`: number (timestamp)
- `to`: number (timestamp)
- `limit`: number
- `offset`: number

Response: 200 OK

Отчёты и Модерация

POST `/api/v1/reports`

Создание отчёта

Headers: `Authorization: Bearer <token>`

Request Body:

```
{
  "targetType": "chat|user|contact|invitation",
  "targetId": "uuid",
  "reason": "spam|harassment|inappropriate|fake|other",
  "description": "string"
}
```

Response: 201 Created

GET /api/v1/reports

Получение списка отчётов (для админов)

Headers: Authorization: Bearer <token>

Query Parameters:

- **status:** "pending|reviewing|resolved|rejected"
- **targetType:** "chat|user|..."

PUT /api/v1/reports/:reportId

Обработка отчёта (для админов)

Request Body:

```
{
  "status": "resolved|rejected",
  "resolution": "string"
}
```

Администрирование

GET /api/v1/admin/users

Получение списка пользователей (админ)

Headers: Authorization: Bearer <token>

Required: Admin role

GET /api/v1/admin/users/:userId

Получение подробной информации о пользователе

PUT /api/v1/admin/users/:userId/role

Изменение роли пользователя

Request Body:

```
{
  "isAdmin": true,
  "isSuperAdmin": false,
}
```

```
"adminRoles": ["users", "chats"]
}
```

DELETE `/api/v1/admin/users/:userId`

Блокировка/удаление пользователя

GET `/api/v1/admin/chats`

Получение списка чатов (админ)

GET `/api/v1/admin/messages`

Получение сообщений (для модерации)

GET `/api/v1/admin/reports`

Получение отчётов (админ)

GET `/api/v1/admin/analytics`

Получение аналитики (админ)

Response: 200 OK

```
{
  "success": true,
  "data": {
    "totalUsers": 1000,
    "activeUsers": 500,
    "totalChats": 5000,
    "totalMessages": 100000,
    "dailyActiveUsers": 200,
    "newUsersToday": 10
  }
}
```

GET `/api/v1/admin/versions`

Получение списка версий приложения

POST `/api/v1/admin/versions`

Добавить новую версию

PUT `/api/v1/admin/versions/:version`

Обновить версию

DELETE /api/v1/admin/versions/:version

Удалить версию

Синхронизация

POST /api/v1/sync/keys

Синхронизация ключей шифрования между устройствами

Headers: Authorization: Bearer <token>

Request Body:

```
{
  "userId": "uuid",
  "deviceId": "string",
  "keys": [
    {
      "keyId": "string",
      "publicKey": "string",
      "encryptedPrivateKey": "string"
    }
  ]
}
```

GET /api/v1/sync/keys

Получение ключей для синхронизации

Headers: Authorization: Bearer <token>

Query: userId=uuid (для другого устройства пользователя)

POST /api/v1/sync/state

Синхронизация состояния (чаты, настройки)

GET /api/v1/sync/state

Получение синхронизированного состояния

WebRTC и Звонки

POST /api/v1/webrtc/offer

Создание WebRTC offer для звонка

POST `/api/v1/webrtc/answer`

Ответ на WebRTC offer

POST `/api/v1/webrtc/ice-candidate`

Обмен ICE кандидатами

POST `/api/v1/calls`

Создание записи о звонке

GET `/api/v1/calls/:callId`

Получение информации о звонке

Статусы (Сторис)

GET `/api/v1/statuses`

Получение статусов (сторис) контактов

Headers: `Authorization: Bearer <token>`

Response: 200 OK

POST `/api/v1/statuses`

Создание статуса

Headers: `Authorization: Bearer <token>`

Content-Type: `multipart/form-data`

Request:

- `file`: File (image/video)
- `type`: "image|video"

Response: 201 Created

DELETE `/api/v1/statuses/:statusId`

Удаление статуса

POST `/api/v1/statuses/:statusId/view`

Отметить просмотр статуса

Поиск

GET /api/v1/global-search

Глобальный поиск (пользователи, чаты, сообщения)

Headers: Authorization: Bearer <token>

Query Parameters:

- **q:** string (search query, required)
- **type:** "users|chats|messages|all"
- **limit:** number

Response: 200 OK

```
{
  "success": true,
  "data": {
    "users": [...],
    "chats": [...],
    "messages": [...],
    "total": 50
  }
}
```

Инсталлер и Обновления

GET /api/v1/installer

Получить информацию для инсталлера PWA

Response: 200 OK

GET /api/v1/versions

Получить информацию о версиях приложений

Response: 200 OK

```
{
  "success": true,
  "data": {
    "web": {
      "version": "1.0.0",
      "minVersion": "0.9.0",
      "updateUrl": "https://..."
    },
    "android": { ... },
    "ios": { ... },
  }
}
```

```
"desktop": { ... }  
}  
}
```

🍷 WebSocket События

Подключение

URL: `wss://api.balloo.app/ws` или `ws://localhost:3001/ws`

Query: `?token=<jwt_token>`

Client → Server

Событие	Описание	Данные
<code>auth</code>	Аутентификация соединения	<code>{ token: string }</code>
<code>subscribe:chat</code>	Подписка на обновления чата	<code>{ chatId: string }</code>
<code>unsubscribe:chat</code>	Отписка от чата	<code>{ chatId: string }</code>
<code>message:send</code>	Отправка сообщения	<code>{ chatId, content, type, ... }</code>
<code>message:read</code>	Подтверждение прочтения	<code>{ chatId, messageId }</code>
<code>typing:start</code>	Начало набора текста	<code>{ chatId }</code>
<code>typing:stop</code>	Прекращение набора	<code>{ chatId }</code>
<code>presence:update</code>	Обновление статуса	<code>{ status: "online" "offline" "away" }</code>
<code>call:offer</code>	WebRTC offer	<code>{ callId, offer }</code>
<code>call:answer</code>	WebRTC answer	<code>{ callId, answer }</code>
<code>call:ice-candidate</code>	ICE кандидат	<code>{ callId, candidate }</code>

Server → Client

Событие	Описание	Данные
<code>auth:success</code>	Успешная аутентификация	<code>{ userId, expiresAt }</code>
<code>auth:error</code>	Ошибка аутентификации	<code>{ code, message }</code>
<code>message:new</code>	Новое сообщение	<code>{ message }</code>
<code>message:updated</code>	Сообщение обновлено	<code>{ messageId, updates }</code>

Событие	Описание	Данные
message:deleted	Сообщение удалено	{ messageId, chatId }
message:read	Сообщение прочитано	{ messageId, readerId }
chat:update	Обновление чата	{ chat }
chat:members:update	Изменились участники	{ chatId, members }
typing:start	Кто-то печатает	{ chatId, userId }
typing:stop	Кто-то прекратил	{ chatId, userId }
presence:update	Изменение статуса	{ userId, status }
notification:new	Новое уведомление	{ notification }
call:offer	Входящий WebRTC offer	{ callId, from, offer }
call:answer	Ответ на звонки	{ callId, answer }
call:ice-candidate	ICE кандидат	{ callId, candidate }
status:new	Новый статус (сторис)	{ status }
status:view	Просмотр статуса	{ statusId, userId }

✖ Формат Ошибок

Все ошибки возвращаются в едином формате:

```
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "Человекочитаемое сообщение",
    "details": {}
  }
}
```

Коды ошибок:

- **UNAUTHORIZED** - Неверный или отсутствующий токен
- **FORBIDDEN** - Недостаточно прав
- **NOT_FOUND** - Ресурс не найден
- **VALIDATION_ERROR** - Ошибка валидации данных
- **INVALID_CREDENTIALS** - Неверные учётные данные
- **TOKEN_EXPIRED** - Токен истёк
- **RATE_LIMITED** - Превышен лимит запросов
- **INTERNAL_ERROR** - Внутренняя ошибка сервера

- **USER_ALREADY_EXISTS** - Пользователь уже существует
- **CHAT_NOT_FOUND** - Чат не найден
- **MESSAGE_NOT_FOUND** - Сообщение не найдено
- **FILE_UPLOAD_ERROR** - Ошибка загрузки файла
- **YANDEX_API_ERROR** - Ошибка Яндекс API

Схема Базы Данных

Таблицы (для встроенной БД)

users

```
id (UUID, PRIMARY KEY)
email (VARCHAR, UNIQUE)
passwordHash (VARCHAR)
displayName (VARCHAR)
fullName (VARCHAR)
avatar (VARCHAR)
publicKey (TEXT) -- RSA public key для E2E
provider (VARCHAR) -- 'email' | 'yandex'
yandexId (VARCHAR)
yandexToken (TEXT) -- ENCRYPTED
yandexRefreshToken (TEXT) -- ENCRYPTED
settings (JSONB)
familyRelations (JSONB)
pushTokens (JSONB)
isAdmin (BOOLEAN)
isSuperAdmin (BOOLEAN)
adminRoles (JSONB)
createdAt (TIMESTAMP)
updatedAt (TIMESTAMP)
lastSeen (TIMESTAMP)
```

chats

```
id (UUID, PRIMARY KEY)
type (VARCHAR) -- 'private' | 'group'
name (VARCHAR)
avatar (VARCHAR)
participants (JSONB) -- user IDs
members (JSONB) -- userId -> { role, joinedAt }
adminIds (JSONB)
createdBy (UUID, FOREIGN KEY -> users.id)
description (TEXT)
isFavorite (JSONB) -- userId -> boolean
pinned (JSONB) -- userId -> boolean
unreadCount (JSONB) -- userId -> count
lastMessage (JSONB)
```

```
createdAt (TIMESTAMP)
updatedAt (TIMESTAMP)
```

messages

```
id (UUID, PRIMARY KEY)
chatId (UUID, FOREIGN KEY -> chats.id)
senderId (UUID, FOREIGN KEY -> users.id)
type (VARCHAR) -- 'text' | 'image' | 'video' | 'file' | 'audio'
content (TEXT) -- ENCRYPTED
encryptedInfo (JSONB) -- { ciphertext, iv, authTag, keyId }
attachmentId (UUID)
replyToId (UUID)
forwardFromId (UUID)
reactions (JSONB)
readBy (JSONB) -- user IDs
status (VARCHAR)
edited (BOOLEAN)
editedAt (TIMESTAMP)
createdAt (TIMESTAMP)
```

attachments

```
id (UUID, PRIMARY KEY)
messageId (UUID, FOREIGN KEY -> messages.id)
chatId (UUID, FOREIGN KEY -> chats.id)
uploaderId (UUID, FOREIGN KEY -> users.id)
fileName (VARCHAR)
mimeType (VARCHAR)
fileSize (INTEGER)
yandexDiskPath (VARCHAR)
yandexDiskId (VARCHAR)
publicUrl (VARCHAR)
thumbnailUrl (VARCHAR)
width (INTEGER)
height (INTEGER)
duration (INTEGER)
status (VARCHAR) -- 'uploading' | 'ready' | 'failed'
createdAt (TIMESTAMP)
```

invitations

```
id (UUID, PRIMARY KEY)
code (VARCHAR, UNIQUE)
chatId (UUID, FOREIGN KEY -> chats.id)
invitedBy (UUID, FOREIGN KEY -> users.id)
```

```
maxUses (INTEGER)
usedCount (INTEGER)
expiresAt (TIMESTAMP)
isPermanent (BOOLEAN)
isActive (BOOLEAN)
createdAt (TIMESTAMP)
```

contacts

```
id (UUID, PRIMARY KEY)
userId (UUID, FOREIGN KEY -> users.id)
contactUserId (UUID, FOREIGN KEY -> users.id)
displayName (VARCHAR)
isFavorite (BOOLEAN)
isBlocked (BOOLEAN)
createdAt (TIMESTAMP)
```

notifications

```
id (UUID, PRIMARY KEY)
userId (UUID, FOREIGN KEY -> users.id)
type (VARCHAR)
title (VARCHAR)
body (VARCHAR)
data (JSONB)
read (BOOLEAN)
createdAt (TIMESTAMP)
expiresAt (TIMESTAMP)
```

sessions

```
id (UUID, PRIMARY KEY)
userId (UUID, FOREIGN KEY -> users.id)
refreshToken (VARCHAR)
platform (VARCHAR)
deviceId (VARCHAR)
lastActive (TIMESTAMP)
expiresAt (TIMESTAMP)
```

devices

```
id (UUID, PRIMARY KEY)
userId (UUID, FOREIGN KEY -> users.id)
platform (VARCHAR)
```

```
deviceId (VARCHAR)
pushToken (VARCHAR)
deviceName (VARCHAR)
lastActive (TIMESTAMP)
createdAt (TIMESTAMP)
```

reports

```
id (UUID, PRIMARY KEY)
targetType (VARCHAR)
targetId (UUID)
reportedBy (UUID, FOREIGN KEY -> users.id)
reason (VARCHAR)
description (TEXT)
status (VARCHAR)
reviewedBy (UUID)
reviewedAt (TIMESTAMP)
createdAt (TIMESTAMP)
```

versions

```
id (UUID, PRIMARY KEY)
platform (VARCHAR) -- 'web' | 'android' | 'ios' | 'desktop'
version (VARCHAR)
minVersion (VARCHAR)
updateUrl (VARCHAR)
releaseNotes (TEXT)
isForceUpdate (BOOLEAN)
createdAt (TIMESTAMP)
```

Переменные Окружения

```
# Server
PORT=3001
NODE_ENV=development|production

# JWT
JWT_SECRET=your-super-secret-jwt-key
JWT_EXPIRES_IN=7d
REFRESH_TOKEN_EXPIRES_IN=30d

# Yandex OAuth
YANDEX_CLIENT_ID=your-yandex-client-id
YANDEX_CLIENT_SECRET=your-yandex-client-secret
YANDEX_REDIRECT_URI=http://localhost:3001/api/v1/auth/yandex/callback
```

```
# Yandex Disk
YANDEX_DISK_BASE_URL=https://cloud-api.yandex.net/v1/disk

# Email (for password reset, verification)
EMAIL_HOST=smtp.yandex.ru
EMAIL_PORT=587
EMAIL_USER=your-email@yandex.ru
EMAIL_PASSWORD=your-email-password

# Database (embedded)
DB_TYPE=better-sqlite3|nedb|lowdb
DB_PATH=./data/database.db

# Security
BCRYPT_ROUNDS=12
ENCRYPTION_KEY=your-encryption-key-for-yandex-tokens

# CORS
CORS_ORIGIN=http://localhost:3000

# Rate Limiting
RATE_LIMIT_WINDOW=900000
RATE_LIMIT_MAX_REQUESTS=100
```

Порядок Реализации

Фаза 1: Основное (MVP)

1. ☐ Базовая структура проекта и сервер
2. ☐ База данных и схемы
3. ☐ Регистрация и вход (email/password)
4. ☐ JWT аутентификация
5. ☐ CRUD пользователей
6. ☐ CRUD чатов (private)
7. ☐ Отправка/получение сообщений
8. ☐ WebSocket для реального времени
9. ☐ Уведомления

Фаза 2: Расширенные функции

10. ☐ Яндекс.Авторизация
11. ☐ Яндекс.Диск интеграция
12. ☐ E2E шифрование
13. ☐ Групповые чаты
14. ☐ Роли и права
15. ☐ Приглашительные ссылки
16. ☐ Контакты

17. ☐ Статусы (онлайн/офлайн)

Фаза 3: Дополнительные функции

- 18. ☐ Статусы (сторис)
- 19. ☐ WebRTC звонки
- 20. ☐ Синхронизация ключей
- 21. ☐ Поиск
- 22. ☐ Отчёты и модерация
- 23. ☐ Админ-панель
- 24. ☐ Баллы и награды
- 25. ☐ Инсталлер и обновления

Примечания

- Все даты/времени в Unix timestamp (milliseconds)
- Все ID в формате UUID
- Все шифрованные данные в base64
- Все эндпоинты (кроме auth) требуют Bearer token
- Для файлов используется multipart/form-data
- Пагинация через offset/limit
- E2E шифрование - сообщения шифруются на клиенте
- Сервер передаёт только зашифрованные данные
- Яндекс токены шифруются перед сохранением в БД

Users

GET `/api/v1/users/:id`

Получение публичных данных пользователя

Response: 200 OK

```
{
  "success": true,
  "data": {
    "id": "uuid",
    "username": "string",
    "firstName": "string",
    "lastName": "string",
    "avatar": "string",
    "status": "online|offline|away",
    "lastSeen": "datetime"
  }
}
```

GET /api/v1/users/search

Поиск пользователей

Query Parameters:

- **q** - search query (required)
- **limit** - max results (default: 20, max: 100)
- **offset** - pagination offset

Response: 200 OK

PUT /api/v1/users/me

Обновление данных текущего пользователя

Headers: Authorization: Bearer <token>

Request Body:

```
{
  "firstName": "string",
  "lastName": "string",
  "bio": "string"
}
```

PUT /api/v1/users/me/avatar

Загрузка аватара

Headers: Authorization: Bearer <token>

Content-Type: multipart/form-data

Response: 200 OK

PUT /api/v1/users/me/status

Обновление статуса

Request Body:

```
{
  "status": "online|offline|away|busy"
}
```

PUT /api/v1/users/me/settings

Обновление настроек пользователя

Request Body:

```
{
  "theme": "light|dark|system",
  "notifications": {
    "enabled": true,
    "sound": true,
    "desktop": true
  },
  "privacy": {
    "lastSeen": "all|contacts|none",
    "avatar": "all|contacts|none",
    "status": "all|contacts|none"
  }
}
```

Chats

GET `/api/v1/chats`

Получение списка чатов пользователя

Headers: `Authorization: Bearer <token>`

Query Parameters:

- `limit` - max results
- `offset` - pagination offset
- `type` - filter by type: `private`, `group`, `channel`

Response: 200 OK

```
{
  "success": true,
  "data": {
    "chats": [
      {
        "id": "uuid",
        "type": "private|group|channel",
        "name": "string",
        "avatar": "string",
        "lastMessage": { ... },
        "unreadCount": 0,
        "updatedAt": "datetime"
      }
    ],
    "pagination": { ... }
  }
}
```

```
}  
}
```

GET /api/v1/chats/:chatId

Получение информации о чате

Headers: Authorization: Bearer <token>

Response: 200 OK

POST /api/v1/chats

Создание нового чата

Headers: Authorization: Bearer <token>

Request Body:

```
{  
  "type": "private|group",  
  "participantIds": ["uuid", "uuid"], // для private  
  "name": "string", // для group  
  "avatar": "string" // для group  
}
```

PUT /api/v1/chats/:chatId

Обновление чата

Headers: Authorization: Bearer <token>

Request Body:

```
{  
  "name": "string",  
  "avatar": "string",  
  "description": "string"  
}
```

DELETE /api/v1/chats/:chatId

Удаление/выход из чата

Headers: Authorization: Bearer <token>

Response: 200 OK

GET `/api/v1/chats/:chatId/members`

Получение списка участников чата

Response: 200 OK

POST `/api/v1/chats/:chatId/members`

Добавление участника в чат

Request Body:

```
{
  "userId": "uuid"
}
```

DELETE `/api/v1/chats/:chatId/members/:userId`

Удаление участника из чата

PUT `/api/v1/chats/:chatId/read`

Отметить чат как прочитанный

Request Body:

```
{
  "lastMessageId": "uuid"
}
```

Messages

GET `/api/v1/chats/:chatId/messages`

Получение истории сообщений

Query Parameters:

- **limit** - max results (default: 50)
- **before** - get messages before this messageId
- **after** - get messages after this messageId

Response: 200 OK

```
{
  "success": true,
```

```

"data": {
  "messages": [
    {
      "id": "uuid",
      "chatId": "uuid",
      "senderId": "uuid",
      "sender": { ... },
      "content": "string",
      "type": "text|image|video|file|system",
      "attachments": [...],
      "replyTo": { ... },
      "reactions": [...],
      "isEdited": false,
      "createdAt": "datetime",
      "updatedAt": "datetime"
    }
  ],
  "hasMore": false
}

```

POST /api/v1/chats/:chatId/messages

Отправка сообщения

Headers: Authorization: Bearer <token>

Content-Type: multipart/form-data или application/json

Request Body (text):

```

{
  "content": "string",
  "type": "text",
  "replyTo": "messageId (optional)"
}

```

Request Body (with attachments):

```

{
  "content": "string (optional)",
  "type": "image|video|file",
  "attachments": ["fileIds"]
}

```

Response: 201 Created

PUT /api/v1/messages/:messageId

Редактирование сообщения

Request Body:

```
{  
  "content": "string"  
}
```

DELETE `/api/v1/messages/:messageId`

Удаление сообщения

Query Parameters:

- `forEveryone` - boolean, delete for all or just for sender

POST `/api/v1/messages/:messageId/reactions`

Добавление реакции к сообщению

Request Body:

```
{  
  "emoji": "string"  
}
```

DELETE `/api/v1/messages/:messageId/reactions/:emoji`

Удаление реакции

PUT `/api/v1/messages/:messageId/read`

Подтверждение прочтения сообщения



Contacts

GET `/api/v1/contacts`

Получение списка контактов

Headers: `Authorization: Bearer <token>`

Response: 200 OK

POST `/api/v1/contacts`

Добавление контакта

Request Body:

```
{
  "userId": "uuid"
}
```

DELETE `/api/v1/contacts/:userId`

Удаление контакта

GET `/api/v1/contacts/requests`

Получение запросов в друзья

Query Parameters:

- `type` - `received` | `sent`

POST `/api/v1/contacts/requests`

Отправка запроса в друзья

Request Body:

```
{
  "userId": "uuid",
  "message": "string (optional)"
}
```

PUT `/api/v1/contacts/requests/:requestId`

Обработка запроса в друзья

Request Body:

```
{
  "action": "accept|reject"
}
```

Files & Media

POST `/api/v1/files/upload`

Загрузка файла

Headers: Authorization: Bearer <token>

Content-Type: multipart/form-data

Request:

- **file** - file to upload
- **type** - image|video|audio|document (optional)

Response: 201 Created

```
{
  "success": true,
  "data": {
    "id": "uuid",
    "url": "string",
    "type": "string",
    "size": 12345,
    "name": "string",
    "mimeType": "string"
  }
}
```

GET /api/v1/files/:fileId

Получение информации о файле

DELETE /api/v1/files/:fileId

Удаление файла

GET /api/v1/files/:fileId/download

Скачивание файла

Notifications

GET /api/v1/notifications

Получение уведомлений пользователя

Headers: Authorization: Bearer <token>

Query Parameters:

- **type** - filter by type
- **read** - filter by read status
- **limit** - max results
- **offset** - pagination

Response: 200 OK

PUT /api/v1/notifications/:notificationId/read

Отметить уведомление как прочитанное

PUT /api/v1/notifications/read-all

Отметить все уведомления как прочитанные

DELETE /api/v1/notifications/:notificationId

Удалить уведомление

Groups (Special Chats)

POST /api/v1/groups

Создание группы

Request Body:

```
{
  "name": "string",
  "description": "string",
  "avatar": "string",
  "participantIds": ["uuid", ...],
  "settings": {
    "onlyAdminsCanPost": false,
    "onlyAdminsCanAddMembers": false
  }
}
```

GET /api/v1/groups/:groupId

Получение информации о группе

PUT /api/v1/groups/:groupId

Обновление группы

DELETE /api/v1/groups/:groupId

Удаление группы (только админ)

PUT /api/v1/groups/:groupId/admins

Назначение администраторов

WebSocket Events

Реальные события для WebSocket подключения:

Client → Server

- **auth** - аутентификация соединения
- **subscribe:chat** - подписка на обновления чата
- **unsubscribe:chat** - отписка от чата
- **message:send** - отправка сообщения
- **message:read** - подтверждение прочтения
- **typing:start** - начало набора текста
- **typing:stop** - прекращение набора
- **presence:update** - обновление статуса

Server → Client

- **auth:success** - успешная аутентификация
- **auth:error** - ошибка аутентификации
- **message:new** - новое сообщение
- **message:updated** - сообщение обновлено
- **message:deleted** - сообщение удалено
- **chat:update** - обновление чата
- **chat:members:update** - изменились участники чата
- **typing:start** - кто-то начал печатать
- **typing:stop** - кто-то прекратил печатать
- **presence:update** - изменение статуса пользователя
- **notification:new** - новое уведомление

Примечания

- Все эндпоинты (кроме auth) требуют Bearer токен в заголовке **Authorization**
- Все даты в формате ISO 8601
- Ошибки возвращаются в стандартном формате:

```
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "Человекочитаемое сообщение",
    "details": {}
  }
}
```

```
}  
}
```

- Пагинация использует **offset** и **limit**
- Для файлов используются CDN или объектное хранилище